🎨 Chetan's Action Plan: Frontend & Visualization Implementation

Hello Chetan bhai! You are the Frontend and Visualization Engineer for this hackathon. The backend boys (Parth and Pawan) are handling the data, but the judges will only judge what they *see*. Your job is to make this ontology graph look ekdum premium, interactive, and mind-blowing.

Since we are short on time, we will put all our React code into a single, powerful `App.jsx` file to avoid complex imports. Let's crack this!

Step 1: Scaffold the React App (Hour 1)

We will use Vite instead of Create React App because it is super fast.

1. Open your terminal (outside the backend folder) and run:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install axios react-force-graph-2d lucide-react
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

2. Open `tailwind.config.js` and update the `content` array so Tailwind knows where to look for classes:

```
// tailwind.config.js
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

3. Open `src/index.css` and replace everything with these Tailwind directives:

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen,
  background-color: #0f172a; /* Dark background looks best for graphs */
  color: white;
```

```
}
```

Step 2: Build the UI Shell & Graph (Hour 2-6)

Instead of making 10 different components and wasting time, let's put our Sidebar, State Management, and Graph Visualization all inside `src/App.jsx`.

Open `src/App.jsx`, delete the default Vite code, and paste this master code.

Read the comments carefully, bhai. They explain how the APIs are connected!

```
// src/App.jsx
import React, { useState, useEffect, useRef } from 'react';
import axios from 'axios';
import ForceGraph2D from 'react-force-graph-2d';
import { Search, Activity, Database, Loader2 } from 'lucide-react';

const BACKEND_URL = 'http://localhost:5000/api'; // Make sure Parth's server is r

export default function App() {
  const [topic, setTopic] = useState('');
  const [graphData, setGraphData] = useState({ nodes: [], links: [] });
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState('Enter a topic to build the knowledge gr
  const fgRef = useRef();

  // 1. Fetch existing graph data when the app loads
  useEffect(() => {
    fetchGraph();
  }, []);

  const fetchGraph = async () => {
    try {
      const response = await axios.get(\`\${BACKEND_URL}/graph\`);
      setGraphData(response.data);
    } catch (error) {
      console.error("Error fetching graph data:", error);
      setMessage("Backend se connect nahi ho pa raha bhai. Check if server is run
    }
  };

  // 2. Trigger Ingestion Pipeline (Pawan + Parth's code)
  const handleIngest = async (e) => {
    e.preventDefault();
    if (!topic) return;

    setLoading(true);
    setMessage(\`Fetching global news for "\${topic}" & extracting AI ontology...

    try {
      // This calls Parth's POST API
      await axios.post(\`\${BACKEND_URL}/ingest\`, { topic });
      setMessage("Success! New intelligence added to the graph.");
      setTopic('');
      // Fetch the updated graph
      fetchGraph();
```

```
    } catch (error) {
      console.error("Ingestion error:", error);
      setMessage("Kuch gadbad ho gayi processing mein.");
    } finally {
      setLoading(false);
    }
  };

  // 3. Logic to color nodes based on their type
  const getNodeColor = (node) => {
    switch(node.group) {
      case 'Location': return '#3b82f6';     // Blue
      case 'Organization': return '#10b981'; // Green
      case 'Person': return '#f59e0b';       // Yellow
      case 'Concept': return '#ec4899';      // Pink
      default: return '#9ca3af';             // Gray
    }
  };

  return (
    <div className="flex h-screen w-full bg-slate-900 overflow-hidden text-white

      {/* SIDEBAR */}
      <div className="w-80 bg-slate-800 border-r border-slate-700 p-6 flex flex-c
        <div className="flex items-center gap-3 mb-8">
          <Activity className="text-blue-400" size={32} />
          <h1 className="text-xl font-bold leading-tight">Global Ontology Engine<
        </div>

        <form onSubmit={handleIngest} className="flex flex-col gap-4 mb-8">
          <div>
            <label className="text-sm text-slate-400 mb-1 block">Analyze New Topi
            <div className="relative">
              <input
                type="text"
                value={topic}
                onChange={(e) => setTopic(e.target.value)}
                placeholder="e.g. Oil Prices, Taiwan..."
                className="w-full bg-slate-900 border border-slate-700 rounded-lg
                disabled={loading}
              />
              <Search className="absolute left-3 top-2.5 text-slate-500" size={18
            </div>
          </div>

          <button
            type="submit"
            disabled={loading || !topic}
            className="w-full bg-blue-600 hover:bg-blue-700 disabled:bg-slate-700
          >
            {loading ? <Loader2 className="animate-spin" size={18} /> : <Database
            {loading ? 'Processing via AI...' : 'Ingest & Analyze'}
          </button>
        </form>

        <div className="bg-slate-900 p-4 rounded-lg border border-slate-700 text-
          <p>{message}</p>
        </div>

        <div className="mt-auto">
          <h3 className="text-sm font-semibold text-slate-400 mb-3 uppercase trac
```

```
            <div className="flex flex-col gap-2 text-sm">
              <div className="flex items-center gap-2"><span className="w-3 h-3 rou
              <div className="flex items-center gap-2"><span className="w-3 h-3 rou
              <div className="flex items-center gap-2"><span className="w-3 h-3 rou
              <div className="flex items-center gap-2"><span className="w-3 h-3 rou
            </div>
          </div>
        </div>

        {/* GRAPH CANVAS AREA */}
        <div className="flex-1 relative cursor-grab active:cursor-grabbing">
          {graphData.nodes.length === 0 && !loading ? (
            <div className="absolute inset-0 flex items-center justify-center text-
              No data in the graph yet. Ingest a topic to start!
            </div>
          ) : (
            <ForceGraph2D
              ref={fgRef}
              graphData={graphData}
              nodeLabel="id"
              nodeColor={getNodeColor}
              nodeRelSize={6}
              linkColor={() => '#475569'}
              linkWidth={1.5}
              linkDirectionalArrowLength={4}
              linkDirectionalArrowRelPos={1}
              linkDirectionalParticles={2}
              linkDirectionalParticleSpeed={0.01}
              onNodeClick={(node) => {
                // Smooth zoom animation when a node is clicked!
                fgRef.current.centerAt(node.x, node.y, 1000);
                fgRef.current.zoom(8, 2000);
              }}
            />
          )}
        </div>
      </div>
    );
  }
```

Step 3: Run and Test (Hour 7)

1. Start your React app:

   ```
   npm run dev
   ```

2. Make sure Parth has his backend running (`node server.js` on port 5000).
3. Open `http://localhost:5173` (or whatever port Vite gives you) in your browser.
4. If the database already has data, the graph will load automatically and the nodes will fly into place (physics engine magic! ✨).

5. Type "Global Chip Shortage" in the input and click the blue button. You should see the loading spinner, and after 10-15 seconds, the graph should update with new nodes dynamically!

Step 4: Demo Rehearsal (Hour 8-9)

Chetan, the hackathon pitch relies heavily on your screen. Practice this flow:

1. Start with an empty database (ask Parth to clear Neo4j).
2. Show the judges the blank dark screen.
3. Type a trending topic (e.g., "Middle East Crisis").
4. Click **Ingest & Analyze**. Explain that the backend is fetching live news and the AI is extracting an ontology.
5. Watch the judges' faces when the nodes suddenly appear and connect themselves on the canvas.